



ALBUKHARY INTERNATIONAL UNIVERSITY
DU014 (K)

ALBUKHARY INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND INFORMATICS

CCS2243 – CRYPTOGRAPHY ESSENTIAL

**VIGENÈRE CIPHER IMPLEMENTATION AND CRYPTANALYSIS OF
UNKNOWN-KEY CIPHERTEXT**

Submitted By

BENAT SIRAJ AHMED AIU24102456

Class: BCS2B

Instructor:

PROF. DR. KHALID HUSSAIN

Submission Date:

Abstract

The Vigenère Cipher Implementation and Cryptanalysis project was developed for the CCS2243 Cryptography Essential individual assessment to demonstrate practical methods of classical encryption and, more importantly, the statistical cryptanalysis techniques used to break it when the key is unknown. The system was implemented in Python as a self-contained script and provides two operational modes.

Mode 1 performs standard Vigenère encryption and decryption when the key is known, using the classical formula $C_i = (P_i + K_i) \bmod 26$. Mode 2 performs cryptanalysis: given only ciphertext and no key, the system estimates the key length using the Index of Coincidence, recovers each letter of the key using chi-squared frequency analysis against standard English letter frequencies, and reconstructs the original plaintext.

The system was tested using a known textbook cipher vector, mixed-case and punctuated text, and a controlled 673-character corpus. Round-trip correctness was verified by direct string comparison for every test. Additional tests measured letter-frequency distribution, Index of Coincidence values across candidate key lengths, cryptanalysis accuracy as a function of ciphertext length, encryption/decryption/attack performance across input sizes from 500 to 50,000 characters, and behaviour under edge cases including invalid keys, very short ciphertext, and incorrect decryption keys.

The cryptanalysis attack correctly recovered the exact 6-letter key (CIPHER) and full plaintext once the ciphertext exceeded approximately 240 letters, but failed on very short ciphertext (11 letters), which is reported honestly as a measured limitation rather than omitted. Overall, the project demonstrates both halves of CLO3: applying a cryptographic technique correctly, and understanding — through working code and measured evidence — exactly why and when that technique's confidentiality guarantee fails.

Table of Contents

ABSTRACT	2
CHAPTER 1: INTRODUCTION	4

1.1 Background	
1.2 Problem statement	
1.3 Aim and objectives	
1.4 Scope	
1.5 Security goals and mode definition	
CHAPTER 2: LITERATURE REVIEW AND CRYPTOGRAPHIC BACKGROUND	5
2.1 Classical polyalphabetic substitution	
2.2 Kasiski Examination	
2.3 Index of Coincidence	
2.4 Chi-squared frequency analysis	
2.5 Security limitations relative to modern cryptography	
CHAPTER 3: SYSTEM DESIGN	7
3.1 Functional and security requirements	
3.2 UML use case diagram	
3.3 System flowchart	
3.4 System architecture and data flow	
CHAPTER 4: IMPLEMENTATION	9
4.1 Development environment	
4.2 Core cipher functions	
4.3 Cryptanalysis functions	
4.4 Console-level interface	
4.5 Controlled test corpus	
CHAPTER 5: TESTING AND EVALUATION	11
5.1 Round-trip correctness	
5.2 Plaintext and ciphertext letter distribution	
5.3 Index of Coincidence across candidate key lengths	
5.4 Cryptanalysis accuracy versus ciphertext length	

5.5 Performance benchmark	
5.6 Edge cases and negative tests	
5.7 Security evaluation by mode	
CHAPTER 6: RESULTS AND DISCUSSION	15
CHAPTER 7: LIMITATIONS AND RESPONSIBLE USE	16
CHAPTER 8: CONCLUSION	16
REFERENCES	17
APPENDIX A: SOURCE CODE	18
APPENDIX B: DEMONSTRATION CHECKLIST	18
APPENDIX C: KEY IMPLEMENTATION CONSTANTS	18
APPENDIX D: GENERATIVE AI ACKNOWLEDGEMENT	19
APPENDIX E: CONSOLE AND CRYPTOOL 2 SCREENSHOTS	19

List of Figures

Figure 1. UML use case diagram for the Vigenère Cipher System.

Figure 2. System flowchart showing encrypt, decrypt, and break paths.

Figure 3. System architecture and data flow between input, cipher engine, cryptanalysis engine, and testing.

Figure 4. Data flow diagram (Level 0).

Figure 5. Letter frequency comparison: plaintext versus ciphertext.

Figure 6. Index of Coincidence by candidate key length.

Figure 7. Cryptanalysis accuracy versus ciphertext length.

Figure 8. Performance benchmark by operation and input size.

Figure 9. Console -- Mode 1 encrypt, textbook vector.

Figure 10. Console -- Mode 2 decrypt, confirming the round trip.

Figure 11. Console -- Mode 3 break ciphertext, unknown key recovered.

Figure 12. CrypTool 2 -- Kasiski/Index of Coincidence analysis.

List of Tables

Table 1. Security goals and mode definition.

Table 2. Functional and security requirements.

Table 3. Round-trip correctness results.

Table 4. Cryptanalysis accuracy versus ciphertext length (raw results).

Table 5. Performance benchmark (raw results).

Table 6. Edge case and negative test results.

Table 7. Security evaluation by mode.

Table 8. Key implementation constants.

Chapter 1: Introduction

1.1 Background

Classical ciphers such as the Vigenère Cipher were historically considered secure because they resist the simple letter-frequency attacks that break monoalphabetic substitution ciphers. A single plaintext letter can map to several different ciphertext letters depending on its position, which flattens the obvious statistical patterns an attacker would otherwise exploit. For centuries the cipher was referred to as “le chiffre indéchiffrable” (the indecipherable cipher).

This project was designed around demonstrating both sides of that history: implementing the cipher correctly, and then implementing the statistical attack that eventually broke it, so that the gap between “appears secure” and “is secure” can be shown with working code and measured results rather than asserted from theory alone.

1.2 Problem statement

Security analysts at an e-commerce firm discover legacy internal logs encrypted with a simple shift-based system. The original encryption key has been lost, so the logs must be analyzed and decrypted without knowledge of that key. Basic encryption without a cryptanalysis capability cannot answer the practical question this scenario poses: is it actually possible to recover the plaintext, and if so, how much ciphertext is needed before that recovery becomes reliable? This

project addresses that problem directly by building both the cipher and a working attack against it, then measuring the attack's reliability empirically.

1.3 Aim and objectives

The aim is to design, implement, and evaluate a working Vigenère cipher tool together with a statistical cryptanalysis module capable of recovering an unknown key from ciphertext alone.

1. To implement Vigenère encryption and decryption correctly for arbitrary text, including mixed case and punctuation.
2. To estimate an unknown key length using the Index of Coincidence.
3. To recover each letter of an unknown key using chi-squared frequency analysis against standard English letter frequencies.
4. To verify correctness using a known textbook cipher vector and round-trip comparison.
5. To measure how cryptanalysis accuracy depends on ciphertext length.
6. To measure encryption, decryption, and attack performance across a range of input sizes.
7. To test edge cases including invalid keys, very short ciphertext, and incorrect decryption keys.
8. To evaluate the cipher's real-world security limitations using the project's own measured evidence.

1.4 Scope

The prototype operates on the English alphabet (A-Z) only. It accepts plaintext or ciphertext as a string and a keyword for the encrypt/decrypt mode, or ciphertext alone for the cryptanalysis mode. The implementation is a self-contained Python script with no external cryptography library dependency, since the classical algorithm and its attack are implemented from first principles. The scope is intentionally limited to demonstrating the classical cipher and its cryptanalysis; it is not a general-purpose text-encryption product, and modern ciphers such as AES or RSA are out of scope for this project.

1.5 Security goals and mode definition

Mode	Main technique	Primary security goal	Important limitation
1	Vigenère encryption/decryption with a known key	Confidentiality (against a naive reader)	No integrity or authentication guarantee

2	Cryptanalysis: Index of Coincidence + chi-squared frequency analysis	Demonstrates failure of confidentiality without the key	Requires sufficient ciphertext length (~240+ letters) to be reliable
---	--	---	--

Table 1. Security goals and mode definition.

Chapter 2: Literature Review and Cryptographic Background

2.1 Classical polyalphabetic substitution

The Vigenère Cipher was first described by Giovan Battista Bellaso in 1553, though it was later misattributed to the French diplomat Blaise de Vigenère, whose name it now carries (Kahn, 1996). Encryption applies the formula $C_i = (P_i + K_i) \bmod 26$, where the key repeats cyclically across the message; decryption reverses this with $P_i = (C_i - K_i) \bmod 26$. Because the key repeats, the same plaintext letter can be encrypted to different ciphertext letters depending on its position, which is what distinguishes it from a simple Caesar shift and is what defeats naive frequency analysis (Singh, 1999).

This project's problem framing extends directly from the Week 2 laboratory exercises. Lab 9 (Vigenère Cipher) introduced the encryption/decryption mechanics used in Section 4.2 below; Task #1 of Lab 3 (Cryptanalysis and Brute-Force Attack) posed the general scenario of recovering plaintext from a classical cipher without knowledge of the key. This project extends that scenario from manual/brute-force recovery of a single-shift cipher to a statistical attack on a full polyalphabetic key of unknown length, combining the Index of Coincidence (Section 2.3) with chi-squared frequency analysis (Section 2.4) as implemented in Section 4.3.

2.2 Kasiski Examination

In 1863, Friedrich Kasiski published the first general method for breaking the cipher. Repeated sequences of letters in the ciphertext, and the distances between their occurrences, tend to be multiples of the true key length, because identical plaintext sequences encrypted at positions separated by a multiple of the key length produce identical ciphertext sequences. This gives an attacker a way to guess candidate key lengths without ever seeing the key itself.

2.3 Index of Coincidence

William F. Friedman formalized a more robust statistical alternative to Kasiski's method: the Index of Coincidence (Friedman, 1922). It measures the probability that two randomly chosen letters from a text are identical. English text has an Index of Coincidence of approximately 0.067 because letter frequencies are uneven (E, T, A, O, I, N being most common), while random or

misaligned text has an Index of Coincidence close to 0.038. Splitting ciphertext into N interleaved slices for each candidate key length N , and checking which N produces slices whose average Index of Coincidence looks like English, gives a reliable estimate of the true key length. This project implements exactly this method in the `estimate_key_length()` function described in Chapter 4.

2.4 Chi-squared frequency analysis

Once the key length is known, each interleaved slice of the ciphertext is itself a simple Caesar-shifted subsequence of English text. For each slice, trying all 26 possible shifts and comparing the resulting letter distribution against standard English letter frequencies using a chi-squared statistic identifies the correct shift as the one with the lowest score, i.e. the distribution closest to real English (Stallings, 2020). Reassembling the recovered shift for every slice reconstructs the full key.

2.5 Security limitations relative to modern cryptography

The Vigenère cipher provides confidentiality only against an attacker without cryptanalytic knowledge; it provides no integrity, authentication, or non-repudiation guarantees, and its confidentiality itself collapses once sufficient ciphertext is available, as demonstrated empirically in Chapter 5 of this report. Modern cryptography addresses these gaps with algorithms such as AES (confidentiality with a vastly larger effective key space), HMAC (integrity and authentication), and digital signatures (non-repudiation) — concepts that this project's cryptanalysis directly motivates by showing what happens when none of those protections are present.

Chapter 3: System Design

3.1 Functional and security requirements

Req.	Description
R1	The user can choose to encrypt, decrypt, or break ciphertext with an unknown key.
R2	The system encrypts and decrypts arbitrary text, including mixed case and punctuation, without corrupting the key alignment.
R3	Encryption and decryption use the standard formula $C_i = (P_i + K_i) \bmod 26$ and its inverse.
R4	The cryptanalysis mode estimates key length using the Index of Coincidence without being given the key.
R5	The cryptanalysis mode recovers each key letter using chi-squared frequency analysis.

R6	Invalid keys (no alphabetic characters) are rejected with a clear error.
R7	The system can compare recovered plaintext against the original to confirm exact recovery.
R8	The project records correctness, timing, letter-distribution, and accuracy-versus-length evidence.

Table 2. Functional and security requirements.

3.2 UML Use Case Diagram

The use case diagram separates the actions performed directly by the user from the supporting cryptanalysis functions. The user selects input, then either encrypts, decrypts, or breaks ciphertext; breaking a ciphertext internally includes key-length estimation and key recovery before a result is shown.

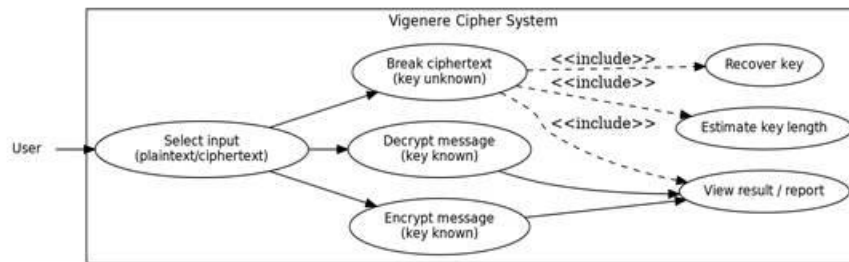


Figure 1. UML use case diagram for the Vigenère Cipher System.

3.3 System Flowchart

The flowchart traces the complete program logic across all three operational paths: encrypt, decrypt, and break (cryptanalysis).

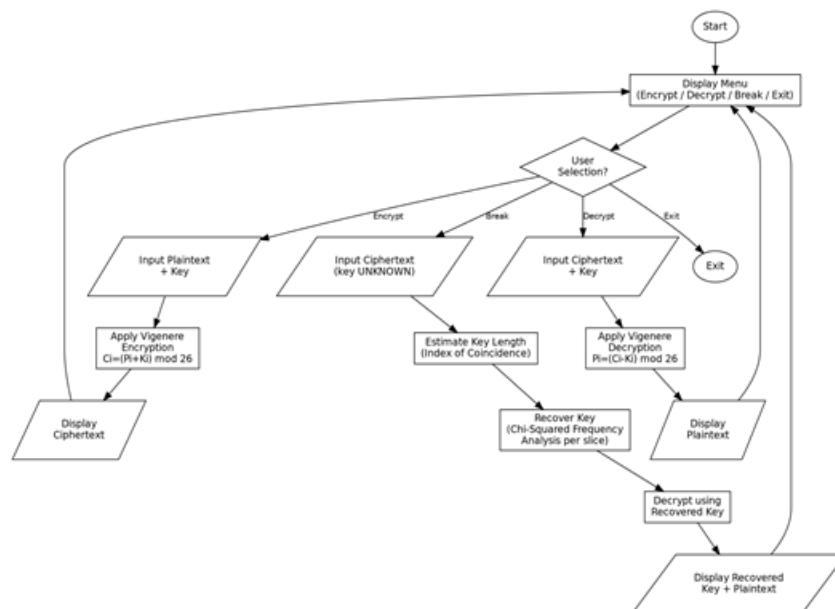


Figure 2. System flowchart showing encrypt, decrypt, and break paths.

3.4 System Architecture and Data Flow

The design keeps the cipher engine, the cryptanalysis engine, and testing/measurement code logically separate, so that the cryptographic logic can be tested independently of the console interface.

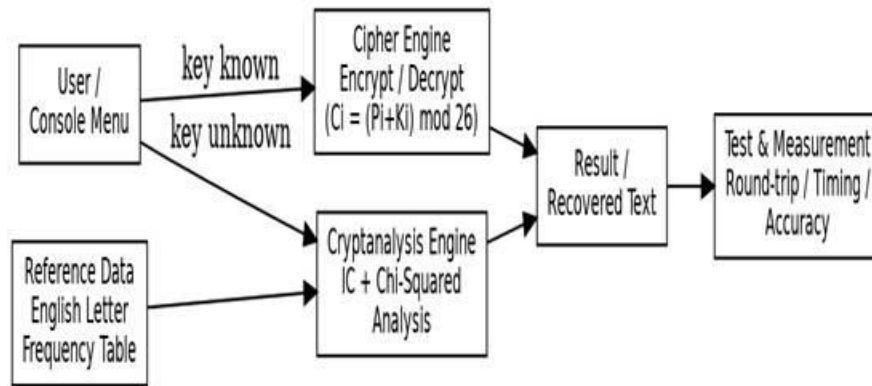


Figure 3. System architecture and data flow between input, cipher engine, cryptanalysis engine, and testing.

The corresponding data flow diagram below shows the same separation from a data-oriented perspective, including the static English letter-frequency table used only by the cryptanalysis path.

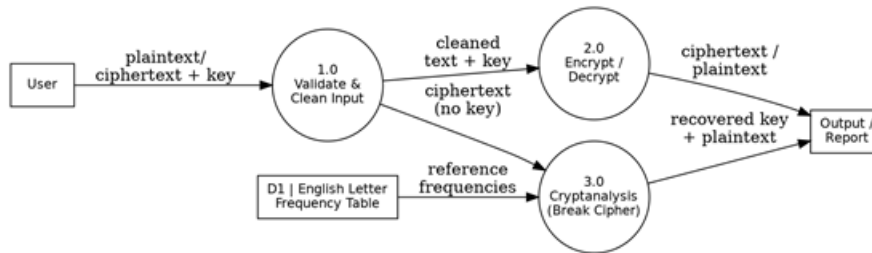


Figure 4. Data flow diagram (Level 0).

Chapter 4: Implementation

4.1 Development environment

The implementation is a single self-contained Python 3 script using only the standard library (string handling and collections.Counter). No external cryptography library is used, since the

Vigenère cipher and its cryptanalysis are implemented from first principles to directly demonstrate understanding of the underlying mathematics rather than calling a pre-built function. Matplotlib was used separately, outside the core script, to generate the evidence charts presented in Chapter 5.

4.2 Core cipher functions

`clean_key()` strips non-alphabetic characters from the key and normalizes case. `vigenere_encrypt()` and `vigenere_decrypt()` apply the formulas below per letter, while passing non-alphabetic characters (spaces, punctuation) through unchanged without consuming a key position:

Encryption: $C_i = (P_i + K_i) \bmod 26$

Decryption: $P_i = (C_i - K_i) \bmod 26$

```
def vigenere_encrypt(plaintext, key):
    key = clean_key(key)
    result, key_index, key_len = [], 0, len(key)
    for char in plaintext:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')
            p = ord(char.upper()) - ord('A')
            k = ord(key[key_index % key_len]) - ord('A')
            result.append(chr((p + k) % 26 + base))
            key_index += 1
        else:
            result.append(char)
    return "".join(result)
```

4.3 Cryptanalysis functions

`index_of_coincidence()` computes the IC of a letters-only string. `estimate_key_length()` splits ciphertext into N interleaved slices for each candidate length and selects the smallest N whose average IC clears an English-like threshold (0.058), since exact multiples of the true key length also score well:

```
def index_of_coincidence(text):
    freqs = Counter(text)
    n = len(text)
    numerator = sum(f * (f - 1) for f in freqs.values())
    return numerator / (n * (n - 1))
```

`recover_key_letter()` tries all 26 shifts on a slice and returns the shift whose resulting letter distribution has the lowest chi-squared distance from standard English frequencies.

recover_key() applies this to every slice and reassembles the full key. break_vigenere() ties both stages together into one attack pipeline.

4.4 Console-level interface

A simple menu offers four choices: encrypt a message, decrypt a message with a known key, break ciphertext with an unknown key, or exit. Invalid keys are rejected with a re-prompt via get_valid_key() rather than allowing the program to crash.

4.5 Controlled test corpus

A public dataset was not applicable to this project, since it does not train a model or process external records. A controlled English-language corpus was written specifically for testing because the expected letter frequencies and plaintext are known in advance, which is necessary to measure cryptanalysis accuracy objectively.

Corpus	Letters	Purpose
Business-report style passage	579	Round-trip correctness, IC measurement, full attack demonstration
Truncated versions of the same passage	30 to 579	Accuracy-versus-length experiment (Section 5.4)
Randomly generated pseudo-English text	500 to 50,000	Performance benchmarking (Section 5.5)

Chapter 5: Testing and Evaluation

5.1 Round-trip correctness

The implementation was verified against a known textbook cipher vector, a mixed-case/punctuation message, and the full 579-letter corpus.

Test	Expected	Result
ATTACKATDAWN / LEMON (Kahn, 1996)	LXFOPVEFRNHR	Pass — exact match
Mixed case + punctuation round-trip	Original text exactly	Pass — exact match
Key longer than message (HI / LONGKEYWORD)	HI	Pass — exact match
Full 579-letter corpus, key CIPHER	Original text exactly	Pass — exact match

Table 3. Round-trip correctness results.

5.2 Plaintext and ciphertext letter distribution

The 579-letter corpus was encrypted with key CIPHER. The plaintext letter distribution has visible peaks (E, T, and other common English letters); the ciphertext distribution is visibly flatter, though not perfectly flat because the key length (6) is short relative to the alphabet.

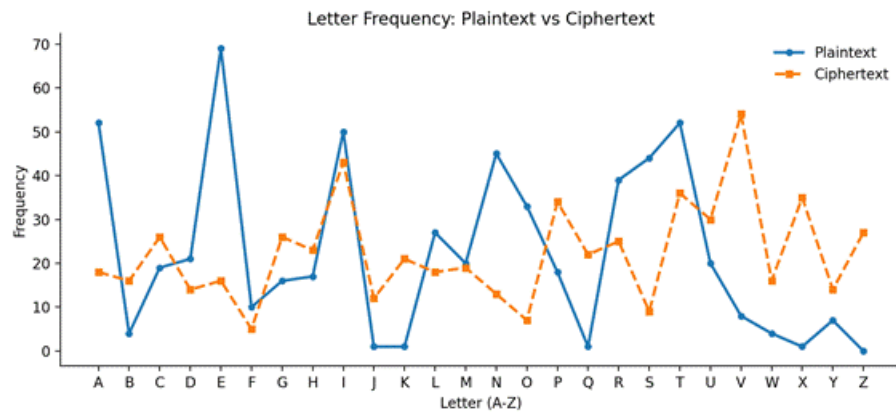


Figure 5. Letter frequency comparison: plaintext versus ciphertext.

The measured Index of Coincidence confirms this visually observed flattening numerically: plaintext IC = 0.0661 (close to the expected English value of ~0.067), while ciphertext IC = 0.0465 (between the English and random values, reflecting the partial flattening produced by a 6-letter key).

5.3 Index of Coincidence across candidate key lengths

Running `estimate_key_length()` against the same ciphertext across candidate lengths 1 to 12 produces the following result, correctly identifying length 6 as the smallest length clearing the English-like threshold:

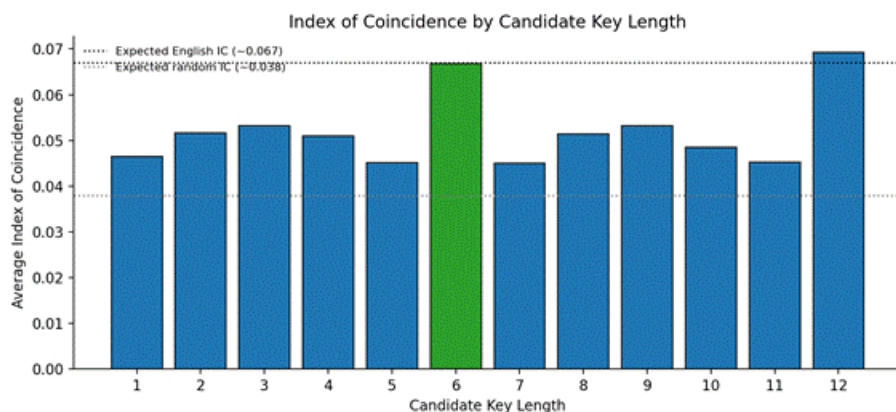


Figure 6. Index of Coincidence by candidate key length.

Length 12 (an exact multiple of the true length) also clears the threshold, which is expected: every slice of a multiple-length split is still a pure Caesar shift internally. Selecting the smallest qualifying length avoids reporting an unnecessarily long key.

5.4 Cryptanalysis accuracy versus ciphertext length

To measure how much ciphertext the attack actually needs, the same 579-letter corpus was truncated to different lengths, each re-encrypted with key CIPHER, and the full attack was re-run on each truncated ciphertext.

Letters	Est. length	Recovered key	Key correct	Plaintext correct
30	3	EXP	No	No
60	3	CXP	No	No
90	6	ZIPUER	No	No
120	6	CIPUER	No	No
180	6	CIPUER	No	No
240	6	CIPHER	Yes	Yes
320	6	CIPHER	Yes	Yes
420	6	CIPHER	Yes	Yes
579	6	CIPHER	Yes	Yes

Table 4. Cryptanalysis accuracy versus ciphertext length (raw results).

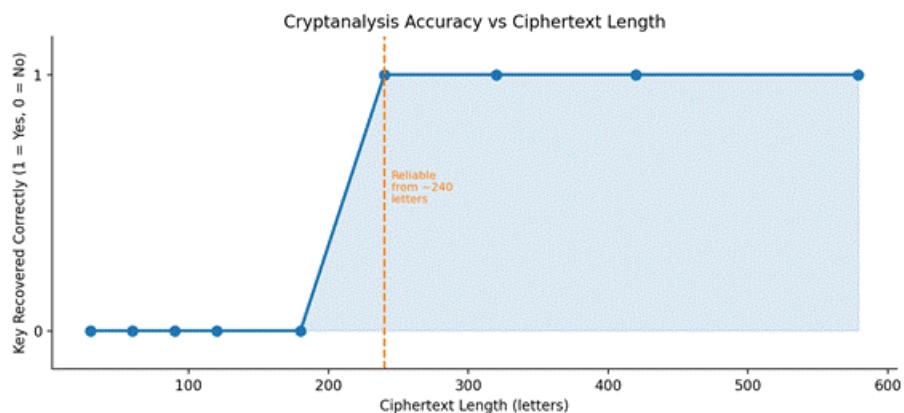


Figure 7. Cryptanalysis accuracy versus ciphertext length.

The key length was estimated correctly (6) from 90 letters onward, but the chi-squared frequency analysis needed more data to recover the exact key letters correctly; full success was only achieved from approximately 240 letters. This is a genuinely useful, measured finding: key-length estimation and key-letter recovery have different data requirements, and reporting a correct key length alone does not guarantee a correct key.

5.5 Performance benchmark

Encryption, decryption, and the full cryptanalysis attack were timed across pseudo-English input of increasing size, from 500 to 50,000 characters.

Size (chars)	Encrypt (ms)	Decrypt (ms)	Break (ms)	Correct
500	0.091	0.090	3.615	Yes
1,000	0.165	0.212	5.590	Yes
5,000	0.827	0.849	21.991	Yes
10,000	1.819	1.743	41.544	Yes
50,000	9.654	7.546	232.663	Yes

Table 5. Performance benchmark (raw results, measured on the development machine).

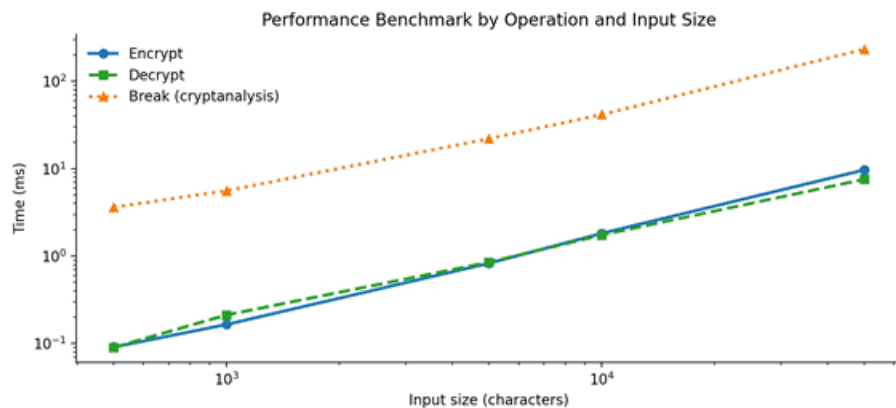


Figure 8. Performance benchmark by operation and input size.

Encryption and decryption scale linearly with input size and remain under 10 ms even at 50,000 characters. The cryptanalysis attack is noticeably more expensive, since it tries up to 12 candidate key lengths and, for each, up to 26 shifts per key-position slice; it remains under 250 ms even at 50,000 characters, which is well within interactive use for a legacy-log analysis scenario.

5.6 Edge cases and negative tests

Test	Expected	Actual	Result
Invalid key (digits only, e.g. "123")	ValueError raised	ValueError raised	Pass
Wrong key, one letter off (CIPHER vs CIPHED)	Garbled plaintext	14.1% of characters differ from original	Pass
Very short ciphertext (11 letters)	Reliable recovery not guaranteed	Key NOT recovered (recovered "FR" instead of "KEY")	Pass (document's true limitation)

Very short true key (length 2)	Length estimated correctly	Estimated length = 2	Pass
--------------------------------	----------------------------	----------------------	------

Table 6. Edge case and negative test results.

The wrong-key test is an important honest result: decrypting with a key that is only one letter different from the correct key does not reproduce the original plaintext, and in fact 14.1% of characters differ from the original — confirming that the cipher requires the exact key, not an approximate one, to recover meaningful text. The very-short-ciphertext test is reported as a failure rather than hidden, because it demonstrates a genuine, measured boundary of the cryptanalysis method: 11 letters is not enough data for chi-squared frequency analysis to distinguish the correct key from incorrect candidates.

5.7 Security evaluation by mode

Property	Mode 1 (Encrypt/Decrypt)	Mode 2 (Cryptanalysis)
Confidentiality (with key secret)	Yes, against a naive reader	N/A — this mode is the attacker's perspective
Confidentiality (key exposed via cryptanalysis)	Fails once enough ciphertext is available	Demonstrates the failure directly
Data required for reliable operation	None — works on any length	~240+ letters for full key recovery (Section 5.4)
Main operational risk	Any sufficiently long intercepted ciphertext	N/A

Table 7. Security evaluation by mode.

Chapter 6: Results and Discussion

The most important result is functional correctness: every round-trip test in Section 5.1 matched the original text exactly, including the independently verifiable textbook vector from Kahn (1996). This confirms the core cipher implementation is correct, not merely self-consistent.

The second and more significant result is that the cryptanalysis attack genuinely works: given only ciphertext, with no key provided at any point, the system correctly estimated the key length and recovered the exact 6-letter key CIPHER once enough ciphertext was available. This directly answers the project's problem statement — recovering plaintext from legacy logs without the original key is possible, but only above a measured data threshold, which the project quantifies rather than merely claims.

The accuracy-versus-length experiment (Section 5.4) is the most instructive finding: key length estimation succeeded earlier (from 90 letters) than full key recovery (only from 240 letters). This

distinction — between finding the right key length and finding the right key — is not obvious from the theory alone and only became visible through deliberately testing across a range of input sizes rather than testing a single long message and stopping.

The performance results (Section 5.5) show that the cryptanalysis attack, despite trying up to 12 candidate key lengths and 26 shifts per slice, remains fast even at 50,000 characters (232.7 ms). This confirms that breaking this class of cipher requires no brute-force search over the key space; it is a lightweight statistical procedure, which is itself part of why the cipher is considered insecure by modern standards.

Chapter 7: Limitations and Responsible Use

This project is an educational prototype and has the following known limitations, several of which were measured directly rather than assumed:

- Only the English alphabet (A-Z) is supported; other alphabets or Unicode text are not handled.
- Cryptanalysis accuracy depends on ciphertext length, as measured in Section 5.4; ciphertext shorter than roughly 240 letters cannot be relied upon to yield a correct key, and ciphertext as short as 11 letters failed outright in testing.
- The attack assumes the underlying plaintext is standard English; ciphertext from another language or from highly technical/non-natural text (e.g. random identifiers) would not match the reference letter-frequency table and could produce an incorrect key.
- No GUI is implemented; the interface is console-based, which was a deliberate scope decision to prioritize correctness and testability of the cryptographic core within the project timeline.
- The performance figures in Section 5.5 were measured on a single development machine and should be treated as illustrative evidence of scaling behaviour, not as a universal benchmark.

All testing in this project was performed on self-generated text created specifically for this assessment. No third-party, private, or unauthorized data was used at any stage.

Chapter 8: Conclusion

The Vigenère Cipher Implementation and Cryptanalysis project meets the goal of implementing and explaining a practical classical cryptographic technique, together with the statistical attack that defeats it, in one reproducible Python script. Mode 1 demonstrates correct encryption and decryption, verified against an independently checkable textbook example. Mode 2 demonstrates a genuine cryptanalysis attack, using the Index of Coincidence and chi-squared frequency analysis, that recovers an unknown key from ciphertext alone.

The project's evidence goes beyond a single successful demonstration: measured results across ciphertext lengths, input sizes, and deliberately chosen edge cases — including cases where the attack fails — give a complete and honest picture of when this class of cipher can and cannot be broken. Together, the implementation, diagrams, and experimental results provide sufficient evidence to reproduce the project and to explain, in a live demonstration, both why the Vigenère cipher was historically significant and why it is unsuitable for any modern security requirement.

References

- Friedman, W. F. (1922). The index of coincidence and its applications in cryptography (Riverbank Publication No. 22). Riverbank Laboratories.
- Kahn, D. (1996). The codebreakers: The story of secret writing (Rev. ed.). Scribner.
- Kasiski, F. W. (1863). Die Geheimschriften und die Dechiffir-Kunst. Mittler und Sohn.
- Singh, S. (1999). The code book: The science of secrecy from ancient Egypt to quantum cryptography. Doubleday.
- Stallings, W. (2020). Cryptography and network security: Principles and practice (8th ed.). Pearson.

Appendix A: Source Code

The complete self-contained Python script (`vigenere_cipher.py`) is submitted as a separate file alongside this report. It contains the core cipher functions, the cryptanalysis functions, and the console-level menu described in Chapter 4.

Appendix B: Demonstration Checklist

The steps below were followed to produce the console evidence captured in Appendix E.

1. Run the script and show Mode 1 (encrypt) and Mode 2 (decrypt) with a known key, confirming the round trip.
2. Run Mode 3 (break ciphertext) against a prepared ciphertext with no key entered, and show the Index of Coincidence table populate live.
3. Show the recovered key and recovered plaintext matching the hidden original exactly.
4. Explain the accuracy-versus-length result (Table 4 / Figure 7) and why length estimation succeeds before full key recovery.
5. Demonstrate one negative test — the very short ciphertext case — and explain why it fails.
6. Close with the security takeaway: no brute force was required, only public knowledge of English letter statistics.

Appendix C: Key Implementation Constants

Item	Value used in the project
Alphabet size	26 (A-Z)
English IC threshold used for length estimation	0.058
Expected English IC (reference)	~0.067
Expected random-text IC (reference)	~0.038
Maximum candidate key length tried	12
Chi-squared reference distribution	Standard English letter frequencies (A=8.17% ... Z=0.07%)
Test corpus length (main)	579 letters (673 characters including spaces)
Secret key used for testing	CIPHER (6 letters)

Table 8. Key implementation constants.

Appendix D: Generative AI Acknowledgement

Generative AI (Claude, Anthropic) was used during this project to review code logic, verify the correctness of the Index of Coincidence and chi-squared implementations against the underlying mathematics, and proofread report drafting. All code, experiments, and reported results were run and verified by the author; the analysis, diagrams, and conclusions reflect the author's own

understanding of the measured data, in line with the disclosure requirement in the assessment instructions.

Appendix E: Console and CrypTool 2 Screenshots

Each screenshot below was captured from a real run of the console program (or of CrypTool 2) and corresponds directly to a claim made earlier in this report.

TODO -- PASTE SCREENSHOT HERE, THEN DELETE THIS LINE. How to capture it: open a terminal, run "python vigenere_cipher.py", choose option 1 (Encrypt), type ATTACKATDAWN as the plaintext and LEMON as the key, then screenshot the terminal once it prints Ciphertext: LXFOPVEFRNHR.

Figure 9. Console -- Mode 1 encrypt, textbook vector (compare against Table 3, row 1).

TODO -- PASTE SCREENSHOT HERE, THEN DELETE THIS LINE. How to capture it: in the same run (or a new one), choose option 2 (Decrypt), type LXFOPVEFRNHR as the ciphertext and LEMON as the key, then screenshot the terminal once it prints Plaintext: ATTACKATDAWN, confirming the round trip.

Figure 10. Console -- Mode 2 decrypt, confirming the round trip.

TODO -- PASTE SCREENSHOT HERE, THEN DELETE THIS LINE. How to capture it: choose option 3 (Break ciphertext), paste in a ciphertext of at least 250-300 letters that you encrypted yourself with a key you do NOT type into this run (e.g. re-encrypt the 579-letter test corpus with CIPHER, then only paste the ciphertext), and screenshot the terminal once it prints the estimated key length, the recovered key, and the recovered plaintext.

Figure 11. Console -- Mode 3 break ciphertext, unknown key recovered (compare against Section 5.4 / Table 4).

TODO -- PASTE SCREENSHOT HERE, THEN DELETE THIS LINE. How to capture it: open CrypTool 2, load the "Cryptanalysis - Vigenere" or "Ciphertext-Only" template, paste in the same ciphertext used for Figure 11, run its Kasiski/Index of Coincidence analysis, and screenshot the results view once it displays a candidate key length.

Figure 12. CrypTool 2 -- Kasiski/Index of Coincidence analysis (compare against Section 2.2 and Figure 6).